Browser 3D Gaussian Splatting: Open-Vocabulary Segmentation, Editing, Mesh Export, and Object Stills

Technical report · implementation note
Gaussian Splatting Web Viewers — WebGPU path
2 September 2026 · revised 3 September 2026 (reciprocal multi-view association, real-GPU profile, topology repair and 3MF)

PDF: open-vocab-3dgs-imagine-pipeline-paper.pdf · operator notes: pipeline.md

---

Abstract

This report describes a *modular* browser pipeline that (i) loads and rasterizes a 3D Gaussian Splatting (3DGS) scene with WebGPU, (ii) captures calibrated views, (iii) obtains promptable masks with SAM 2.1 running in JavaScript through transformers.js and ONNX Runtime Web, (iv) lifts and associates those masks into per-Gaussian instance labels, (v) edits and exports instances, and (vi) fuses depth maps of either an isolated instance or the complete visible scene into a TSDF. The geometry path writes a coloured binary glTF 2.0 (`.glb`) or remeshes with marching tetrahedra, validates/repairs topology, scales to millimetres and writes a 3MF package. A parallel vision–language path supplies open-vocabulary names and Grok Imagine Image 2.0 object-card *edits*. The design follows the 2D-lift paradigm of PointGS, GALA, and LangSplat: **semantics are applied to images composited from known Gaussians under known cameras**, not to unconditioned image generations. A successful topology gate is still not proof of adequate wall thickness, slicer settings or a physically printed object. This is an implementation note for the `Gaussian-Splatting-WebViewers` repository; it does not claim new reconstruction metrics.

Keywords: 3D Gaussian Splatting, WebGPU, SAM 2.1, transformers.js, open-vocabulary segmentation, TSDF, surface nets, marching tetrahedra, GLB, 3MF, Grok Imagine

---

1. Introduction

3D Gaussian Splatting [1] represents a scene as anisotropic 3D Gaussians and rasterizes them with EWA splatting [2] and spherical-harmonic view-dependent color. Official training output is an INRIA `point_cloud.ply` (float covariance, SH degree 3). Compact `.splat` files [6] discard SH rest bands and quantize rotations.

Browser viewers historically packed everything to 32-byte rows and looked like point clouds. This project's WebGPU viewer keeps float Gaussians and SH0–3. Its semantic and geometry layers:

1. Segment captured 3DGS frames with **SAM 2.1 in the browser**, or with a sidecar mask backend.
2. Lift multi-view masks into stable labels on Gaussian indices and expose instance editing/export.
3. Reconstruct an isolated instance or the complete visible scene from orbit depth maps as a coloured GLB or topology-gated, millimetre-scale 3MF mesh.
4. Tag captured frames with **Grok 4.6 vision**, cluster names, and use **Grok Imagine Image 2.0** only to edit crops into object cards under `img_output/`.

The scientific constraint, taken from PointGS [10] and related open-vocab 3DGS work [11–15], is:

> Pixels used for semantics must be produced by the Gaussian image-formation model (or a camera-conditioned repair of it). Unconditioned image generators cannot back-project a mask onto Gaussian index i

.

---

2. Related work (what we reuse vs. what we do not ship)

Table 1 distinguishes methods whose *equations or formats* are in the running system from methods whose *ideas* informed the pipeline but whose code is not vendored. Section 8 lists licenses.

Line of work	Role in this pipeline
Kerbl et al. 3DGS [1] and `diff-gaussian-rasterization` [3]	Image formation, SH eval, 3-sigma Gaussian, $\alpha = o \exp(-\frac{1}{2} r^2)$
Zwicker EWA [2]	Screen-space covariance $JW\Sigma W^\top J^\top$
PointGS [10]	Sparse cloud → 3DGS → SAM on *renders* → distill → kNN back to points. We reuse the **render-then-tag** idea, not their CUDA training
GALA [11], LangSplat [12], Feature3DGS [13], OpenGaussian [14], Gaussian Grouping [15]	Open-vocab / instance fields on 3DGS. Not vendored
ReferSplat [16]	Referring segmentation (sentence → 3D mask). Different product; not implemented
NVIDIA ArtiFixer [17, 18]	Optional camera+opacity video repair. Weights are NVIDIA noncommercial; not in the MIT tree
NVIDIA Fixer / Difix3D+ [19]	Distinct single-step *image* model; do not confuse with ArtiFixer
GaussForge [4]	Multi-format decode IR (PLY, SPLAT, SPZ, KSPLAT, SOG)
antimatter15/splat [6], Kellogg GaussianSplats3D [7], quadjr/aframe-gaussian-splatting [8]	Heritage WebGL viewers and 32-byte layout
xAI Grok vision + Imagine Image 2.0 [20, 21]	Tagging and object-card *edits*
SAM 2.1 [26] + transformers.js [32]	Promptable masks on independent browser-rendered views; the SAM video-memory path is not used
TSDF fusion [28, 29], surface nets [30, 31], marching tetrahedra [37], glTF 2.0 [34], 3MF Core [35]	Isolated splat instance → fused implicit surface → coloured GLB or topology-gated 3MF mesh

---

3. System overview

Default scene: `splats/alarm_clock_generated.splat` (compact 32-byte SH0, 262 144 Gaussians, approximately 8 MB). Serve the **repository root** on port 8090 so the worker can import `../shared/splat-io.js`. The sidecar on 8766 reads `XAI_API_KEY` from the repo-root `.env`. The browser never sees API keys.

3.1 Flowchart

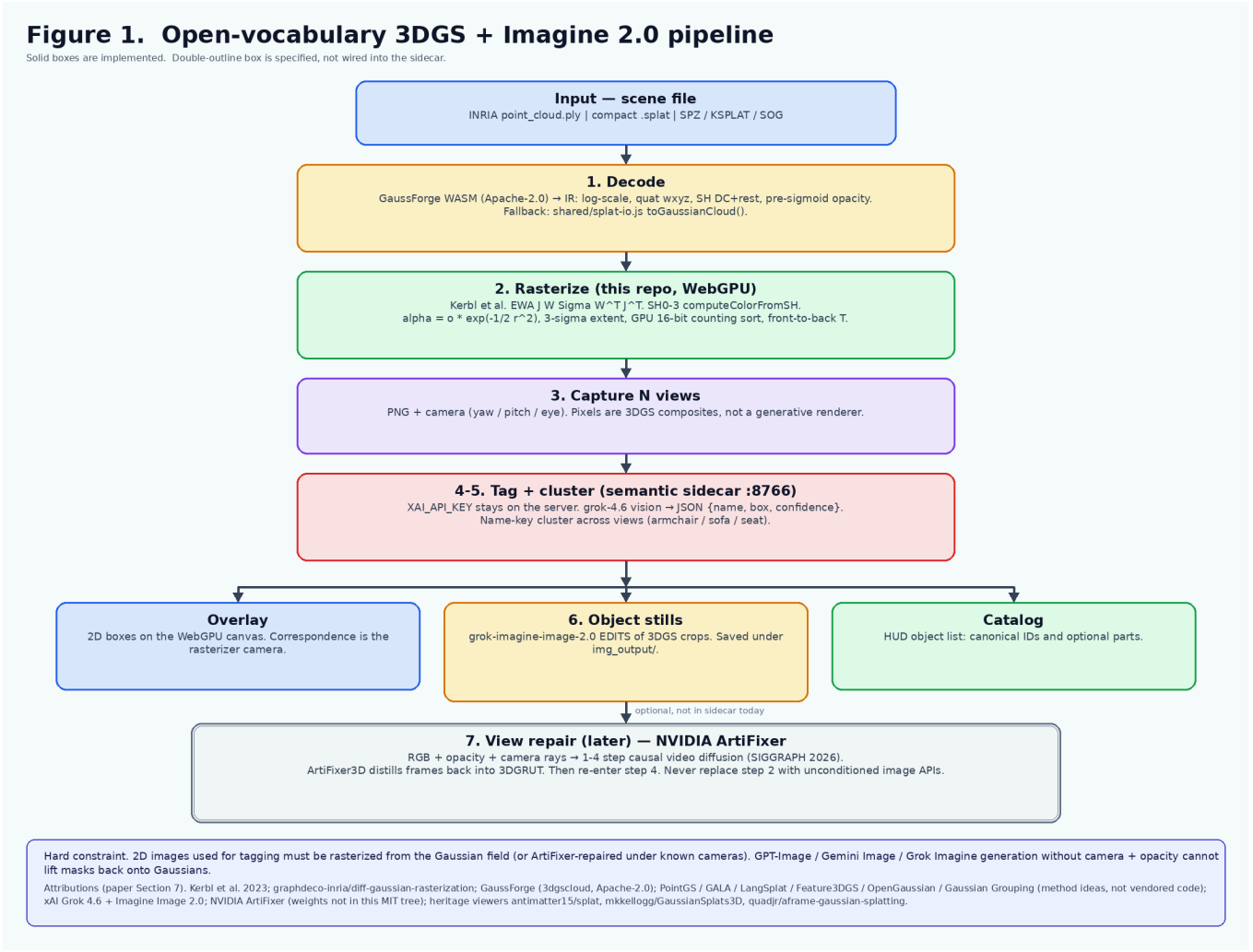


Figure 1. End-to-end workflow. Solid boxes are implemented. ArtiFixer (double outline) is specified but not wired into the sidecar.

```
flowchart TD
  A["Scene file<br/>INRIA PLY / .splat / SPZ / KSPLAT / SOG"] --> B["1
Decode<br/>GaussForge WASM IR<br/>fallback splat-io.js"]
  B --> C["2 Rasterize WebGPU<br/>EWA + SH0–3 + GPU sort"]
  C --> D["3 Capture N views<br/>PNG + camera"]
  D --> E["4–5 Sidecar :8766<br/>grok-4.6 vision + name cluster"]
  E --> F["Overlay boxes"]
  E --> G["6 Imagine 2.0 EDITS of crops<br/>img_output/"]
  E --> H["HUD catalog"]
  C -.-> I["7 Optional ArtiFixer<br/>RGB+opacity+rays → 3DGRUT"]
  I -.-> D
```

3.2 Stages

Stage	Input	Output	Engine	Code
1 Decode	Bytes + filename	`gaussians` $N \times 12$, `sh` $N \times 48$	GaussForge; fallback `toGaussianCloud`	`parse-worker.js`, `shared/splat-io.js`
2 Rasterize	Float cloud	Premultiplied framebuffer	WebGPU, Kerbl image formation	`gpu-renderer.js`
3 Capture	Current / orbit cameras	PNG + yaw/pitch/eye	`snapshotPng`	`main.js`

Stage	Input	Output	Engine	Code
4 Tag	PNG	`{name, box, confidence, parts}`	`grok-4.6` vision	`semantic_sidecar/server.py`
5 Cluster	Per-view tags	Canonical objects	Normalized name key	same
6 Cards	Crop of `best_box`	Studio still	`grok-imagine-image-2.0` **edits**	same; files in `img_output/`
7 Repair (later)	RGB, opacity, rays	Repaired video / 3DGRUT	NVIDIA ArtiFixer	not in sidecar

---

4. Rasterizer (stage 2)

The WebGPU path does **not** pack to 32-byte rows for drawing. Each Gaussian is `pos, opacity, scale, quat(wxyz)` plus 16 RGB SH coefficients.

Following [1, 3]:

- Covariance $\Sigma = RSS^\top R^\top$.
- EWA 2D covariance from Jacobian J of the projective map and view rotation W.
- Color from `computeColorFromSH` (SH_C0…SH_C3), then $+0.5, \max(0, \cdot)$.
- Extent $3\sqrt{\lambda}$ (paper 3-sigma).
- Fragment $\alpha = \min(0.99,\ o\exp(-\frac{1}{2}r^2))$.
- Front-to-back transmittance with blend `(oneMinusDstAlpha, one)` and clear alpha 0.

Compact `.splat` still only recovers SH0; the HUD warns. Full radiance field requires INRIA `point_cloud/iteration_*/point_cloud.ply` with `f_rest_0…44`.

Counting sort is a 16-bit GPU histogram + prefix sum + scatter (near first). It is not copied from a named third-party WebGPU 3DGS engine.

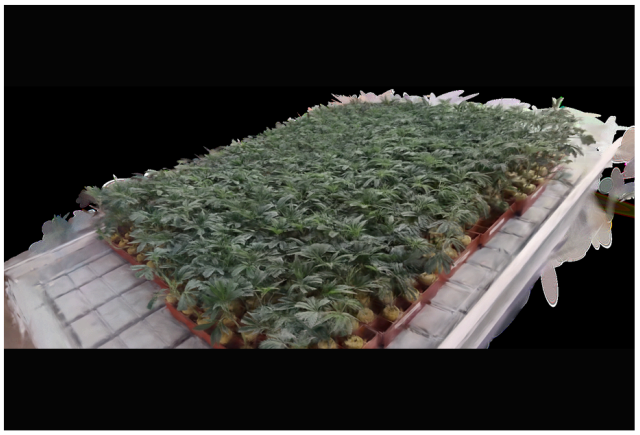
---

5. Semantic sidecar (stages 4–6)

The browser never holds API keys. It POSTs PNG captures to `http://127.0.0.1:8766`.

- **Vision:** `POST https://api.x.ai/v1/chat/completions` model `grok-4.6`, image + JSON schema prompt. Fallback `POST /v1/responses`.
- **Cluster:** lowercase alphanumeric key; merge “armchair / green sofa / seat” by string identity (not CLIP [9]; a later upgrade).
- **Imagine:** `POST https://api.x.ai/v1/images/edits` model `grok-imagine-image-2.0`, `response_format: b64_json`. Prompt asks for a photoreal product still of the **reference crop**. This is image-to-image, not text-to-image from nothing.
- **Library:** `img_output/YYYYMMDD-HHMMSS-name/{source.png, imagine.jpg, meta.json}` plus `img_output/index.jsonl`.

A demo crop of potted plants on a nursery table (Figure 2) was rasterized in the WebGPU viewer and edited with Imagine 2.0. Files: `img_output/20260902-164810-potted-plants/`.



(a) 3DGS crop — potted plants on a nursery table

(b) Imagine 2.0 edit of (a) — object still

Figure 2. (a) WebGPU 3DGS crop of potted plants on a nursery table. (b) Grok Imagine Image 2.0 edit of that crop. The still is an illustration of the tagged object, not a measurement of the Gaussian field.

---

6. Per-Gaussian segmentation, editing and meshing (plan F0–F6)

The tagging pipeline of Sections 3–5 labels *views*. The segmentation plan `plan-segmentacion-edicion-3dgs.md` adds per-Gaussian instances, editing and meshing on top of the same rasterizer, without any per-scene training. Figure 3 shows the data flow; Section 6.1 summarizes the stages, Sections 6.2–6.4 give implementation details and equations, and Section 6.5 records measured results.

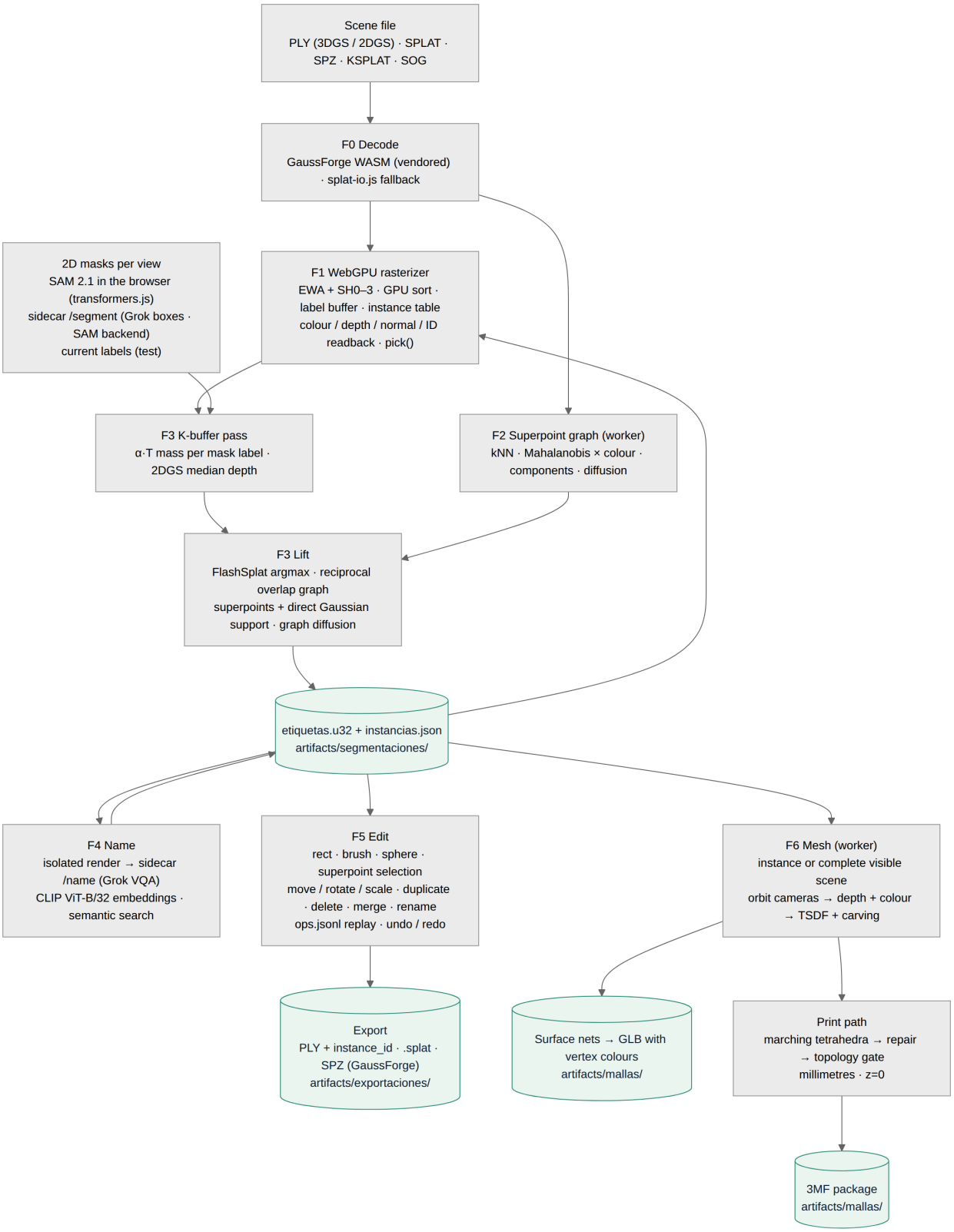


Figure 3. Everything runs in the browser (WebGPU passes and Web Workers); the sidecar only calls xAI for names and persists files under `artifacts/`. *Source* `figures/segmentation-pipeline.mmd`.

6.1 Stages

Phase	Method	Code
F0 Base	GaussForge vendored (offline decode, CDN fallback), 2DGS PLY, `artifacts/`, Node + Playwright test harness (offscreen rendering under SwiftShader)	`vendor/gaussforge/`, `shared/splat-io.js`, `tests/`

Phase	Method	Code
F1 Identity	`u32` label per Gaussian and a 4096-entry instance table (rigid/affine transform, tint, visible/selected) read in the vertex shader; output modes colour / alpha-weighted mean depth / normal / ID with offscreen readback; `pick()` from the ID pass	`gpu-renderer.js`
F2 Superpoints	kNN graph (k = 10, hash grid sorted by cell), symmetric Mahalanobis distance × SH0 colour weights, threshold, connected components → superpoints; weighted-majority label diffusion over the graph	`shared/graph.js` in a worker
F3 Lift	K-buffer fragment pass storing (index, α, depth) per pixel with exact overflow handling; per-Gaussian α·T mass → FlashSplat argmax [23]; reciprocal-best cross-view graph using both superpoint containment and direct Gaussian overlap, with at most one mask from each view per component [24]; LUDVIG-style graph diffusion [25]; export `instancias.json` + `etiquetas.u32`	`contrib-pass.js`, `shared/lift.js`
Masks	SAM 2.1 hiera-tiny in the browser (transformers.js + ONNX Runtime Web, WASM or WebGPU) prompted by projected superpoint centroids with duplicate-mask merging [26]; or the sidecar (`/segment`: Grok boxes rasterised as ellipses, pluggable SAM backend)	`ml-browser.js`, `semantic_sidecar/server.py`
F4 Name	Isolated render per instance (camera framed on its bounds, only its Gaussians) → `/name` Grok VQA with a Spanish JSON schema; CLIP ViT-B/32 image/text embeddings in the browser for semantic search [27]; Imagine card per `id_instancia`	`shared/naming.js`, `ml-browser.js`
F5 Edit / export	Selection on the ID buffer (rectangle, brush, 3D sphere, superpoint), reproducible `ops.jsonl` (assign, transform, delete, duplicate, merge, rename) with undo by replay, transforms baked at export; PLY with `instance_id` / `class_id` / `confidence`, `.splat`, SPZ and compressed PLY through GaussForge; a reloaded PLY restores its instances	`shared/edit-ops.js`, `shared/export-io.js`
F6 Mesh	Selected instance or complete visible scene → Fibonacci orbit → depth/colour → TSDF [28, 29]. Whole-scene scope retains disconnected visible components. GLB uses surface nets [30, 31]. 3MF uses a consistent six-tetrahedra decomposition [37], conservative cleanup/repair, explicit manifold/winding checks, millimetre scaling and OPC packaging [35]	`shared/tsdf.js`, `mesh-ops.js`, `glb.js`, `three-mf.js` in a worker

The invariant identifier is the Gaussian index in the source file; labels, selections, exports and duplicates (`origen`) are expressed on it, and every `instancias.json` is validated against `shared/schemas/instancias.schema.json`.

6.2 SAM 2.1 segmentation in JavaScript

The mask source labelled **SAM 2 (navegador, transformers.js)** is implemented by `BrowserSam` in `gaussian_splatting_webgpu/ml-browser.js`. It is not a separate library named “SAM2.js”. The code lazily imports `@huggingface/transformers` 4.2.0, instantiates `Sam2Model` with the `onnx-community/sam2.1-hiera-tiny-ONNX` fp16 export, and lets ONNX Runtime Web execute on a hardware WebGPU adapter. It falls back to WASM when WebGPU is unavailable, reports a software adapter, or fails to load the model. `scripts/download-ml-models.sh` makes the JavaScript runtime, ONNX Runtime Web files and weights available from the gitignored `vendor/ml/`; otherwise the loader uses jsDelivr and the Hugging Face Hub cache.

Each orbit view is processed independently; this implementation uses SAM 2.1's static-image prompt path, **not** its temporal memory or video tracker. Given an F2 superpoint centroid $\mathbf{c}_s$, the current projection and view matrices map it to clip coordinates

$$\widetilde{\mathbf{q}}_s = PV[\mathbf{c}_s^\top\ 1]^\top = (q_x, q_y, q_z, q_w)^\top, \qquad (x_s, y_s) = \left(W\left(\frac{q_x}{2q_w} + \frac{1}{2}\right),\ H\left(\frac{1}{2} - \frac{q_y}{2q_w}\right)\right).$$

Centroids outside the image are discarded. The offscreen Gaussian-ID pass must also hit a Gaussian from the same superpoint at $(\lfloor x_s \rfloor, \lfloor y_s \rfloor)$; this visibility check prevents occluded centroids from becoming positive prompts. Superpoints are visited by decreasing population, require at least 20 Gaussians, and are capped at 12 prompts per view by default.

The colour render is converted from RGBA to RGB and encoded once per view with `get_image_embeddings`. For each positive point prompt, the ONNX decoder produces three candidate masks and predicted IoU scores; the selected candidate is

$$k^* = \arg\max_{k \in \{1,2,3\}} \widehat{\mathrm{IoU}}_k.$$

A candidate survives only when its score is at least 0.5, its area is at least 64 pixels, and it covers at most 90% of the view. Duplicate prompts on the same object are collapsed using

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \geq 0.8,$$

keeping the higher-score mask. The remaining masks are painted from largest to smallest into one `Uint32Array`, so a smaller foreground object overwrites a larger mask where they overlap. Only then does F3 use the K-buffer and multi-view lift to assign a label to each Gaussian. In other words, SAM supplies 2D evidence; it does not directly segment the 3D file.

The pure mask operations are covered by `tests/unit/ml-browser.test.mjs`. The opt-in `ML_E2E=1` browser test additionally loads the real model, checks two-view SAM lift on the two-sphere scene, and verifies CLIP search; it is kept outside the default suite because it requires downloaded weights and takes minutes under software rendering.

6.3 Splat-to-mesh, GLB and 3MF calculations

F6 does not triangulate Gaussian centres. It frames either one isolated instance or the complete visible scene, renders depth/alpha/colour from multiple cameras, reconstructs an implicit signed-distance field, and extracts its zero level set. The following equations match `shared/naming.js`, `shared/tsdf.js`, `shared/tsdf-worker.js`, `shared/mesh-ops.js`, `shared/glb.js` and `shared/three-mf.js`.

Bounds and orbit. For an axis-aligned instance box with diagonal length d_b, the untransformed bounding-sphere radius is $r = d_b/2$. After an F5 affine transform, the implementation multiplies r by the largest norm of the transform's three linear columns. With vertical field of view ϕ and framing margin $m_f = 1.5$, the orbit distance is

$$d_{\mathrm{cam}} = \max\left(\frac{m_f r}{\sin(\phi/2)},\ 1.05 r \right).$$

For M cameras, a Fibonacci lattice starts with

$$y_i = 1 - \frac{2(i + 1/2)}{M}, \qquad \rho_i = \sqrt{1 - y_i^2}, \qquad \theta_i = i\pi(3 - \sqrt{5}), \qquad \mathbf{d}_i = (\rho_i \cos\theta_i,\ y_i,\ \rho_i \sin\theta_i),$$

for $i = 0, \ldots, M-1$; the implementation clamps pitch to ± 1.3 radians to avoid a degenerate up vector. Each camera looks at the transformed instance centre.

Voxel grid and TSDF. A cube with margin $m_v = 1.15$ has side length $L = 2 r m_v$. At resolution n^3, the sample spacing and default truncation distance are

$$h = \frac{L}{n - 1}, \qquad \mu = 3h.$$

For a world sample $\mathbf{x}$, the view matrix gives camera coordinates $(x_c, y_c, -z)$ with positive depth z. Its nearest depth-map pixel is

$$u = \operatorname{round}\!\left(c_x + f_x \frac{x_c}{z}\right), \qquad v = \operatorname{round}\!\left(c_y - f_y \frac{y_c}{z}\right).$$

If the rendered depth is $D_j(u, v)$ in view j, the signed distance and truncated observation are

$$s_j(\mathbf{x}) = D_j(u, v) - z, \qquad d_j(\mathbf{x}) = \min\!\left(1, \frac{s_j(\mathbf{x})}{\mu}\right), \quad s_j \geq -\mu.$$

Samples farther than μ behind the observed surface are ignored. Otherwise the pixel alpha α_j becomes the observation weight w_j, and the running volume uses the weighted update

$$F' = \frac{WF + w_j d_j}{W + w_j}, \qquad W' = W + w_j, \qquad w_j = \alpha_j.$$

When $\alpha_j < 0.05$, optional empty-space carving instead integrates $d_j = +1$ with weight 0.3. RGB is accumulated with the same alpha weight only within $|s_j| < 0.6\mu$. The default UI uses $M = 24$, $n = 96$, a 256-pixel render edge and either alpha-weighted mean depth or the exact 2DGS median-depth K-buffer.

Surface nets. A voxel cell receives a vertex only if all eight corners have weight at least 0.5 and their TSDF signs differ. For every sign-changing edge (a, b), the zero crossing is linearly interpolated by

$$t_{ab} = \frac{F_a}{F_a - F_b}, \qquad \mathbf{x}_{ab} = \mathbf{x}_a + t_{ab}(\mathbf{x}_b - \mathbf{x}_a).$$

The cell vertex is the mean of its edge crossings. A finite-difference gradient of the eight corner values supplies the outward normal; quads around sign-changing grid edges become two triangles, winding is corrected against those normals, and only the connected component with the most triangles is retained. The diagnostic Euler characteristic is $\chi = V - E + F$; the analytic connected sphere test gives $\chi = 2$, as expected for a closed genus-zero surface.

Print remesh and topology. Surface nets stays the faster GLB path, but ambiguous voxel junctions can place more than two triangles on an edge. For 3MF, each cube is therefore split consistently into six tetrahedra around its body diagonal. A tetrahedron with one or three negative TSDF samples yields one triangle; a two/two sign split yields a quadrilateral split into two triangles. Crossings reuse the same interpolated vertex for each global grid edge, so adjacent cells agree on their boundary [37]. The repair pass then welds positions within $\varepsilon_w = 10^{-6} d_b$ (with a 10^{-9} floor), removes invalid, duplicate and zero-area triangles, propagates consistent face orientation over edge adjacency, fills only simple boundary loops with at most 64 edges, and recomputes vertex normals.

Validation counts the incidence n_e and oriented direction of every undirected mesh edge e. The 3MF gate accepts only finite, non-degenerate, non-duplicate triangles satisfying

$$\forall e:\ n_e = 2, \qquad d_{e,1} = -d_{e,2}, \qquad \left|\frac{1}{6}\sum_{(a,b,c)} \mathbf{p}_a \cdot (\mathbf{p}_b \times \mathbf{p}_c)\right| > 0.$$

Thus boundary edges ($n_e = 1$), non-manifold edges ($n_e > 2$), and equal-direction pairs are reported separately. The Euler characteristic remains diagnostic but is not used as a watertightness test.

Binary glTF. For V mesh vertices and T triangles, the writer stores float32 position, normal and RGB triples plus three uint32 indices per triangle. Its binary payload is therefore

$$B_{\mathrm{BIN}} = 3(3 \cdot 4V) + (3 \cdot 4T) = 36V + 12T \quad \text{bytes}.$$

The GLB adds a 12-byte header, 8-byte JSON-chunk header, padded JSON, and an 8-byte BIN-chunk header [34]:

$$B_{\mathrm{GLB}} = 28 + 4\left\lceil \frac{B_{\mathrm{JSON}}}{4} \right\rceil + 4\left\lceil \frac{B_{\mathrm{BIN}}}{4} \right\rceil.$$

For the current profiled alarm-clock GLB, $V = 12\,419$ and $T = 24\,918$, so $B_{\mathrm{BIN}} = 746\,100$ bytes; the small JSON/chunk overhead yields the observed 747,332-byte file.

6.4 Fabrication handoff: topology-gated 3MF

The implemented print path is **splat → target depth views → TSDF → marching tetrahedra → conservative repair → validation → millimetre scaling → 3MF**, where the target is one isolated instance or every currently visible scene component. Let the repaired bounding-box extents in scene units be $\Delta = (\Delta_x, \Delta_y, \Delta_z)$ and let the requested maximum print dimension be L_{mm}. The viewer computes

$$s_{\mathrm{mm}} = \frac{L_{\mathrm{mm}}}{\max(\Delta_x, \Delta_y, \Delta_z)}, \qquad \mathbf{p}_{\mathrm{mm}} = s_{\mathrm{mm}}(\mathbf{p} - \mathbf{b}_{\min}),$$

where $\mathbf{b}_{\min}$ is the minimum box corner. This makes all coordinates non-negative and places the lowest point on the $z = 0$ build plane. The 3MF model declares `unit="millimeter"`; scene coordinates are not presented as physically calibrated measurements.

`shared/three-mf.js` emits an OPC ZIP package with `[Content_Types].xml`, `_rels/.rels`, and `3D/3dmodel.model`; the root relationship targets the 3D model, whose resources contain one mesh object and whose build section references it, as required by 3MF Core [35]. The package uses uncompressed ZIP entries for deterministic, dependency-free generation and one average base material colour; GLB remains the format that preserves per-vertex colours.

After validation, a closed oriented triangle mesh can provide a solid-volume estimate through signed tetrahedra,

$$V_{\mathrm{mesh}} = \left| \frac{1}{6} \sum_{(a,b,c)} \mathbf{p}_a \cdot (\mathbf{p}_b \times \mathbf{p}_c) \right|, \qquad m_{\mathrm{material}} \approx \rho V_{\mathrm{mesh}},$$

before infill and support corrections. The implementation reports signed volume and surface area but does **not** test self-intersections, minimum wall thickness, overhangs, supports, shrinkage, material/process parameters, slicer compatibility, printer control or a physical part. Accordingly, “topology-gated 3MF” means a closed oriented triangle complex at the requested scale, not a certified printable object.

6.5 Results (software CI and a real RTX 3090 Ti profile)

Measurement	Synthetic two spheres (4 000 Gaussians)	`alarm_clock_generated.splat` (262 144 Gaussians)
F1 depth error at the sphere centre	0.0 % (analytic front surface)	—
F2 graph	2 superpoints in 109 ms	113 154 superpoints in 1.93 s worker / 1.97 s wall on the profiled CPU (threshold 0.3 remains very fine)
F3 K-buffer	exact: 0 mis-assigned Gaussians from one view	760.8 ms median, 937.1 ms p95 at 512 × 320 on RTX 3090 Ti
F3 lift, test masks	6 permuted views → 2 instances, 3D IoU 0.9985 / 1.0 (1.0 / 1.0 after diffusion)	—

Measurement	Synthetic two spheres (4 000 Gaussians)	`alarm_clock_generated.splat` (262 144 Gaussians)
Masks, Grok boxes	—	1 coarse box per view → 5 fragments, 0 cross-view merges
Masks, SAM 2.1 in the browser	2 views → exactly 2 instances of 2 000 Gaussians in the opt-in acceptance	real four-view/eight-prompt run: 20 masks → 13 instances, 7 cross-view merges; 69.29 s total, including 12.24–12.54 s WASM encoding per view
F4 Grok naming	"esfera naranja" 0.92 in 12.6 s; Imagine card 7.7 s	"reloj analógico" 0.72 for the largest instance
F4 CLIP	"an orange ball" / "a blue sphere" rank the right sphere	similarities flat (0.20–0.24) on fragments
F5	export instance → reload shows only that object; `ops.jsonl` replay reproduces the fingerprint	4 986-Gaussian selection moved in 7 ms, SPZ 70 KB in 57 ms; whole scene SPZ 3.7 MB in 1.7 s; undo 0.3 s
F6	GLB radius within ±12 %; instance print acceptance at 8 views / 32^3 → 4 472 vertices, 8 940 triangles, 0 boundary/non-manifold edges, 80 mm maximum, 0.60 MB 3MF; complete-scene GLB and 3MF retain both separated spheres	current clock GLB: 24 views, 96^3 → 12 419 vertices / 24 918 triangles, 747 332 bytes; 1 125.7 ms total (202 ms render, 625 ms fusion, 271 ms extraction/validation); 96^3 3MF correctly blocked on four sub-threshold triangles

The hardware profile used NVIDIA driver 580.173.02, Playwright 1.56.1 / Chromium 141 and the shipped 262 144-Gaussian clock. The original five-run colour and mean-depth medians were 5.0 and 7.7 ms; the post-repair three-run refresh measured 6.5 and 5.0 ms. SAM fell back to WASM because this ONNX WebGPU stack did not expose fp16 support; the renderer remained on the NVIDIA adapter. The 13-instance result is a same-workflow improvement over the earlier greedy 17-instance/2-merge result, but not an accuracy score because no human instance ground truth exists. Raw methodology and reproducible commands are in performance.md.

Tests: 122 Node unit tests and 23 default Playwright tests (plus one opt-in test that runs SAM 2.1 and CLIP in the browser) cover every phase; see testing.md.

6.6 Deviations from the plan and open items

- Meshing runs in JavaScript (TSDF + surface nets / marching tetrahedra) instead of Open3D in the sidecar: it works offline and without a 450 MB dependency; Open3D and Poisson remain optional backends.
- Model weights are not vendored in git; `scripts/download-ml-models.sh` fetches transformers.js, ONNX Runtime Web, SAM 2.1 and CLIP into the gitignored `vendor/ml/`. Without it the browser loads them from jsDelivr and the Hugging Face Hub.
- Reciprocal-best association now combines superpoint evidence with direct Gaussian overlap and improved the clock run from 17 instances/2 merges to 13/7. It still needs labelled multi-view scenes to measure precision/recall and tune thresholds; 113k tiny superpoints remain an over-fragmented prior.
- Vanilla 3DGS gives a noisy mesh (the HUD warns); 2DGS/GOF PLYs are recommended for production meshes. A Chamfer-distance check against a reference mesh is still pending.
- GLB remains a geometry-exchange artifact. 3MF adds repair, an explicit topology gate and requested millimetre scale, but calibration, self-intersection/wall-thickness checks, slicing and physical validation remain external.
- Not implemented: lasso selection, exact baking of non-uniform scales, hole filling after deletion, in-viewer mesh preview, decimation.

---

7. What must not be done

1. Replace stage 2 with GPT-Image, Gemini Image, or Imagine **generation**. Those APIs have no camera or opacity, so correspondence to Gaussians is lost [10].
2. Feed Imagine outputs back into SAM/VLM distillation as if they were 3DGS rasters.
3. Put `XAI_API_KEY` in `main.js`.
4. Vendor ArtiFixer 14B weights into this MIT repository (NVIDIA OneWay Noncommercial license [18]).
5. Confuse ArtiFixer [17] with nvidia/Fixer (Difix3D+) [19].

---

8. Attributions and licenses

This repository's WebGL heritage and the WebGPU/semantic additions use third-party methods and software as follows. **Code copied or linked** is distinguished from **ideas only**.

8.1 Software used in the running system

Component	Origin	License	How we use it
3DGS image formation, SH basis, 3-sigma Gaussian	Kerbl et al. [1]; CUDA reference [3] (`graphdeco-inria/diff-gaussian-rasterization`)	INRIA research license on the CUDA lib; equations reimplemented in WGSL	`gpu-renderer.js`
Multi-format Gaussian IR	GaussForge [4] `@gaussforge/wasm` 0.6.0	Apache-2.0	vendored in `vendor/gaussforge/` (NOTICE.md with SHA-256), jsDelivr only as fallback; decode and SPZ / compressed PLY export
transformers.js 4.2	Hugging Face [32]	Apache-2.0	`ml-browser.js`: SAM 2.1 and CLIP in the browser; from `vendor/ml/` (gitignored, `scripts/download-ml-models.sh`) or jsDelivr
ONNX Runtime Web 1.26	Microsoft	MIT	inference backend of transformers.js (WASM / WebGPU)
SAM 2.1 hiera-tiny (ONNX)	Meta [26]; `onnx-community/sam2.1-hiera-tiny-ONNX`	Apache-2.0	promptable masks per view (weights downloaded, never committed)
CLIP ViT-B/32 (ONNX, q8)	OpenAI [27]; `Xenova/clip-vit-base-patch32`	MIT	per-instance embeddings and text queries (weights downloaded, never committed)
Playwright 1.56	Microsoft	Apache-2.0	e2e tests only (dev dependency)
Marked 18.0.11, KaTeX 0.18.5, `marked-katex-extension` 5.1.12 [36]	respective contributors	MIT	deterministic Markdown + equation rendering for this PDF (dev dependencies)
Pillow	PIL contributors	HPND	sidecar image handling
redis / redis-py	Redis Ltd.	RSALv2/SSPL server (external), MIT client	`project-memory.py` index cache, isolated namespace; not part of the viewer
Mermaid	Mermaid contributors	MIT	figure rendering only (`scripts/render-mermaid.mjs`, not a dependency)

Component	Origin	License	How we use it
PLY / splat fallback parser	this repo `shared/splat-io.js`, layout after [6]	MIT (this repo)	Worker fallback
Compact 32-byte `.splat`	antimatter15/splat [6] (Kevin Kwok)	MIT	Format interop; **not** the WebGPU GPU buffer
Viewer 1	quadjr/aframe-gaussian-splatting [8]	MIT	`gaussian_splatting_1/`
Viewer 2	mkkellogg/GaussianSplats3D [7]	MIT	`gaussian_splatting_2_*`
Grok 4.6 vision, Imagine Image 2.0	xAI API [20, 21]	xAI API terms	Sidecar only
Repo license	Akbar S. / forks	MIT	`LICENSE`

8.2 Methods that informed the design (no code vendored)

Method	Paper / code	Taken from it	Not taken
PointGS [10]	Song, Li, Wang, Yan, CVPR 2026, arXiv:2605.11520	Render 3DGS then tag 2D; do not tag raw point projections	CUDA 3DGS training, SAM contrastive distillation, 2-step ICP
GALA [11]	Alegret, Li, Wang, Liang, Niemeyer, Gasperini, Navab, Tombari, arXiv:2508.14278	Open-vocab 2D+3D on 3DGS; codebook/instance consistency as a *goal*	Scaffold-GS training, dual codebooks
LangSplat [12]	Qin, Li, Zhou, Wang, Pfister, CVPR 2024	Distill language from 2D views of 3DGS	Autoencoder CLIP fields per Gaussian
Feature3DGS [13]	Zhou, Chang, Jiang, Fan, Zhu, Xu, Chari, You, Wang, Kadambi, CVPR 2024	Feature rasterization idea	CNN feature lifting / parallel N-D rasterizer
OpenGaussian [14]	Wu, Meng, Li, Wu, Shi, Cheng, Zhao, Feng, Ding, Wang, Zhang, NeurIPS 2024	Instance clustering + language	Hierarchical CUDA clustering
Gaussian Grouping [15]	Ye, Danelljan, Yu, Ke, ECCV 2024	Lift 2D masks toward 3D instances	Editing GUI
ReferSplat [16]	He, Jie, Wang, Zhou, Hu, Li, Ding, ICML 2025	— (different task: referring 3DGS segmentation)	Referring field training, Ref-LERF
ArtiFixer [17, 18]	de Lutio et al., SIGGRAPH 2026; `nvidia/ArtiFixer`	Opacity-mixing, camera-conditioned repair, 1–4 step AR video; 3DGRUT distill	Weights, Docker training
SPZ / Niantic GaussianCloud [5]	nianticlabs/spz	IR field meanings (log scale, pre-sigmoid alpha, SH layout)	Compression codec (via GaussForge)
2D Gaussian Splatting [22]	Huang et al., SIGGRAPH 2024	Median-depth definition used by the K-buffer resolve; 2DGS PLY reading	Training, normal consistency losses
FlashSplat [23]	Shen et al., ECCV 2024	Closed-form argmax assignment of α·T mass per mask label with a background bias	CUDA implementation
Gaga [24]	Lyu et al., 2024	Cross-view mask association by 3D overlap (here over superpoint histograms)	Training-based refinement
LUDVIG [25]	Marrie et al., 2025	Graph diffusion of labels over Gaussian neighbours	DINOv2 feature uplifting
THGS / superpoint graphs	see plan §2.1	kNN superpoint graph with Mahalanobis × colour weights, no training	Contrastive SAM re-weighting

Method	Paper / code	Taken from it	Not taken
SAM 2 [26]	Ravi et al., 2024	Promptable masks per view (ONNX export by onnx-community)	Video tracking
CLIP [27]	Radford et al., ICML 2021	Image/text embeddings for semantic search	Fine-tuning
Volumetric fusion / KinectFusion [28, 29]	Curless & Levoy 1996; Newcombe et al. 2011	Truncated signed distance integration with weights and space carving	GPU tracking, ICP
Surface nets [30, 31]	Gibson 1998; Lysenko 2012 (naive surface nets)	Vertex per sign-changing voxel, quads across sign-changing edges	Dual contouring with QEF
Marching tetrahedra [37]	Doi and Koide 1991	Consistent six-tetrahedra TSDF remesh for the topology-gated print path	Adaptive refinement
3MF Core [35]	3MF Consortium	OPC package structure, millimetre model, mesh object and build item	Production extensions and slicer policy
SuperSplat [33]	PlayCanvas	Selection-tool UX (rectangle, brush, sphere) and per-object export formats	Its editor and file formats

8.3 License compatibility (summary)

- **This MIT tree** may include GaussForge (Apache-2.0), antimatter15/Kellogg/quadjr (MIT), and a WGSL reimplementation of published 3DGS equations.
- **xAI** is a runtime service: no model weights in git.
- **ArtiFixer weights** must stay out of this repo (NVIDIA OneWay Noncommercial). Code on GitHub is Apache-2.0 but the checkpoint is not.
- **INRIA** `diff-gaussian-rasterization` is not copied; only the published forward formulas [1, 3] are reimplemented.
- **transformers.js (Apache-2.0), ONNX Runtime Web (MIT), SAM 2.1 (Apache-2.0) and CLIP ViT-B/32 (MIT)** are permissive; their weights stay out of git (`vendor/ml/` is gitignored and refilled by `scripts/download-ml-models.sh`). SAM 3 (SAM License) is not used.
- **Playwright, Mermaid, Marked and KaTeX** are development-time only.

---

9. How to run the implemented path

```
cd Gaussian-Splatting-WebViewers
python3 -m http.server 8090 --bind 127.0.0.1
./semantic_sidecar/launch.sh          # :8766
./gaussian_splatting_webgpu/launch-webgpu-chrome.sh
```

Default URL loads `../splats/alarm_clock_generated.splat`. In the HUD: **Tag scene (Grok)** with **Imagine 2.0 object cards** checked. Cards appear on the right and on disk under `img_output/`.

For SH3 radiance fields, drop a trained `point_cloud.ply` instead of the compact splat. Chrome / Vulkan notes: webgpu-chrome.md.

Segmentation, editing and meshing (Section 6): HUD panels **Grupos** → **Segmentación** (mask source *SAM 2 (navegador)* after `scripts/download-ml-models.sh`) → **Instancias** (naming, CLIP search) → **Edición** (selection tools, transforms, export) → **Malla** (GLB, or topology-gated 3MF with a requested maximum dimension in millimetres). Outputs land under `artifacts/`; the HUD walkthrough is in pipeline.md.

---

10. Limitations

- Compact `.splat` is SH0; Imagine stills can look sharper than the 3DGS source because the edit model *hallucinates* high-frequency texture. They are illustrations, not measurements.
- View-tag clustering (stage 5) is string-based; CLIP embeddings [27] are used only for per-instance search (F4).
- SAM 2.1 and CLIP now run in the browser (Section 6); there is still no contrastive Gaussian affinity field and no ArtiFixer GPU in this process.
- Cross-view association has stronger reciprocal Gaussian-overlap evidence, but lacks labelled real-scene precision/recall; F2's default threshold still over-fragments the clock.
- GLB may be noisy or non-manifold. The 3MF gate proves only finite, consistently oriented two-face edge incidence and non-zero signed volume at a requested scale; it does not prove self-intersection freedom, wall thickness, slicer compatibility or a successful physical print (Section 6.4).
- The real-GPU measurements are specific to one RTX 3090 Ti/driver/browser stack. SAM still ran on WASM because the loaded fp16 graph was unsupported by that WebGPU device path.
- Vision may under-label blurry splat views; zoom before tagging.
- Imagine URL downloads can 403; the sidecar requests `b64_json`.

References

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Trans. Graph.*, vol. 42, no. 4, 2023. https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/ Code: https://github.com/graphdeco-inria/gaussian-splatting

[2] M. Zwicker, H. Pfister, J. van Baar, and M. Gross, "EWA Splatting," *IEEE Trans. Visualization and Computer Graphics*, 2002.

[3] GRAPHDECO / Inria, `diff-gaussian-rasterization` (`forward.cu`: `computeColorFromSH`, `computeCov2D`, `computeCov3D`). https://github.com/graphdeco-inria/diff-gaussian-rasterization

[4] 3dgscloud, GaussForge, Apache-2.0. https://github.com/3dgscloud/GaussForge WASM: `@gaussforge/wasm`

[5] Niantic, SPZ / Gaussian cloud IR. https://github.com/nianticlabs/spz

[6] K. Kwok, antimatter15/splat, MIT. https://github.com/antimatter15/splat

[7] M. Kellogg, GaussianSplats3D, MIT. https://github.com/mkkellogg/GaussianSplats3D

[8] K. Kwok and J. Kuwada, quadjr/aframe-gaussian-splatting, MIT. https://github.com/quadjr/aframe-gaussian-splatting

[9] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision" (CLIP), ICML 2021.

[10] Y. Song, Q. Li, W. Wang, and Z. Yan, "PointGS: Semantic-Consistent Unsupervised 3D Point Cloud Segmentation with 3D Gaussian Splatting," CVPR 2026. arXiv:2605.11520. https://github.com/SebastianYIXIAO/pointGS

[11] E. Alegret, K. Li, S. Wang, S. Liang, M. Niemeyer, S. Gasperini, N. Navab, and F. Tombari, "GALA: Guided Attention with Language Alignment for Open Vocabulary Gaussian Splatting," arXiv:2508.14278, 2025.

[12] M. Qin, W. Li, J. Zhou, H. Wang, and H. Pfister, "LangSplat: 3D Language Gaussian Splatting," CVPR 2024. arXiv:2312.16084.

[13] S. Zhou, H. Chang, S. Jiang, Z. Fan, Z. Zhu, D. Xu, P. Chari, S. You, Z. Wang, and A. Kadambi, “Feature 3DGS: Supercharging 3D Gaussian Splatting to Enable Distilled Feature Fields,” CVPR 2024. arXiv:2312.03203.

[14] Y. Wu, J. Meng, H. Li, C. Wu, Y. Shi, X. Cheng, C. Zhao, H. Feng, E. Ding, J. Wang, and J. Zhang, “OpenGaussian: Towards Point-Level 3D Gaussian-based Open Vocabulary Understanding,” NeurIPS 2024. arXiv:2406.02058.

[15] M. Ye, M. Danelljan, F. Yu, and L. Ke, “Gaussian Grouping: Segment and Edit Anything in 3D Scenes,” ECCV 2024. arXiv:2312.00732.

[16] S. He, G. Jie, C. Wang, Y. Zhou, S. Hu, G. Li, and H. Ding, “ReferSplat: Referring Segmentation in 3D Gaussian Splatting,” ICML 2025. arXiv:2508.08252. https://github.com/heshuting555/ReferSplat

[17] R. de Lutio, T. Fischer, Y.-Y. Chang, Y. Zhang, J. Z. Wu, X. Ren, T. Shen, K. Tothova, Z. Gojcic, and H. Turki, “ArtiFixer: Enhancing and Extending 3D Reconstruction with Auto-Regressive Diffusion Models,” SIGGRAPH 2026. https://research.nvidia.com/labs/sil/projects/artifixer/

[18] NVIDIA, `nvidia/ArtiFixer` (weights: NVIDIA OneWay Noncommercial License; Wan 2.1 base: Apache-2.0). https://huggingface.co/nvidia/ArtiFixer Code: https://github.com/nv-tlabs/artifixer

[19] NVIDIA Fixer / Difix3D+, arXiv:2503.01774. https://huggingface.co/nvidia/Fixer — *not* ArtiFixer.

[20] xAI, Grok image understanding (`grok-4.6`). https://docs.x.ai/docs/guides/image-understanding

[21] xAI, Imagine Image 2.0 (`grok-imagine-image-2.0`), generations and edits. https://docs.x.ai/developers/model-capabilities/images/generation https://docs.x.ai/developers/model-capabilities/images/editing

[22] B. Huang, Z. Yu, A. Chen, A. Geiger, S. Gao, “2D Gaussian Splatting for Geometrically Accurate Radiance Fields,” SIGGRAPH 2024. https://surfsplatting.github.io/

[23] Q. Shen, X. Yang, X. Wang, “FlashSplat: 2D to 3D Gaussian Splatting Segmentation Solved Optimally,” ECCV 2024. https://arxiv.org/abs/2409.08270

[24] W. Lyu, X. Li, A. Kundu, Y.-H. Tsai, M.-H. Yang, “Gaga: Group Any Gaussians via 3D-aware Memory Bank,” arXiv:2404.07977, 2024. https://arxiv.org/abs/2404.07977

[25] J. Marrie, R. Ménégaux, M. Arbel, D. Larlus, J. Mairal, “LUDVIG: Learning-free Uplifting of 2D Visual features to Gaussian Splatting scenes,” arXiv:2410.14462, 2025. https://arxiv.org/abs/2410.14462

[26] N. Ravi et al., “SAM 2: Segment Anything in Images and Videos,” arXiv:2408.00714, 2024; ONNX export `onnx-community/sam2.1-hiera-tiny-ONNX`. https://github.com/facebookresearch/sam2

[27] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” ICML 2021; ONNX export `Xenova/clip-vit-base-patch32`. https://github.com/openai/CLIP

[28] B. Curless, M. Levoy, “A Volumetric Method for Building Complex Models from Range Images,” SIGGRAPH 1996.

[29] R. A. Newcombe et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” ISMAR 2011.

[30] S. F. F. Gibson, “Constrained Elastic Surface Nets: Generating Smooth Surfaces from Binary Segmented Data,” MICCAI 1998.

[31] M. Lysenko, “Smooth Voxel Terrain (Part 2)” — naive surface nets, 2012. https://0fps.net/2012/07/12/smooth-voxel-terrain-part-2/

[32] Hugging Face, transformers.js (`@huggingface/transformers`), Apache-2.0. https://github.com/huggingface/transformers.js

[33] PlayCanvas, SuperSplat (MIT). https://github.com/playcanvas/supersplat

[34] Khronos 3D Formats Working Group, "glTF 2.0 Specification," version 2.0.1. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html

[35] 3MF Consortium, "3MF Core Specification," version 1.4.0. https://github.com/3MFConsortium/spec_core

[36] Marked contributors, Marked; KaTeX contributors, KaTeX; Uzi Ashkenazi, `marked-katex-extension` (MIT). https://github.com/markedjs/marked https://github.com/KaTeX/KaTeX https://github.com/UziTech/marked-katex-extension

[37] A. Doi and A. Koide, "An Efficient Method of Triangulating Equi-Valued Surfaces by Using Tetrahedral Cells," *IEICE Transactions*, 1991.

Appendix A. File map

Path	Role
`gaussian_splatting_webgpu/`	Viewer, rasterizer, capture, HUD
`semantic_sidecar/server.py`	Vision + Imagine + `img_output/`
`shared/splat-io.js`	Shared I/O
`gaussian_splatting_webgpu/contrib-pass.js`, `ml-browser.js`	K-buffer pass (F3); SAM 2.1 + CLIP in the browser
`shared/graph.js`, `lift.js`, `naming.js`, `edit-ops.js`, `export-io.js`, `tsdf.js`, `mesh-ops.js`, `glb.js`, `three-mf.js`, `schemas.js` (+ workers)	Superpoints, reciprocal mask association, edit log, TSDF mesh, topology repair, GLB/3MF encoders and schema checks (plan F2–F6)
`shared/schemas/instancias.schema.json`	Schema of `instancias.json` (`scripts/validate-instancias.mjs`)
`scripts/download-ml-models.sh`, `render-mermaid.mjs`, `bench-graph.mjs`, `profile-webgpu.mjs`, `check-source.sh`	Model setup, figures, benchmarks, reproducible profiles and source checks
`artifacts/{segmentaciones,exportaciones,mallas}/`	Generated outputs (gitignored)
`docs/figures/segmentation-pipeline.{mmd,svg,png}`	Figure 3
`docs/plan-segmentacion-edicion-3dgs.md`	Roadmap and per-phase status (Spanish)
`img_output/`	Imagine library (gitignored except README)
`docs/pipeline.md`	Short operator notes
`docs/webgpu-chrome.md`	Chrome/Vulkan launch
`index.html`, `.github/workflows/pages.yml`, `scripts/build-pages.sh`	Public project page, allowlisted artifact build and GitHub Pages deployment
`docs/figures/pipeline-flowchart.png`	Figure 1
`docs/figures/demo-potted-plants-pair.png`	Figure 2
`docs/open-vocab-3dgs-imagine-pipeline-paper.md`	This report (Markdown)
`scripts/render-report-pdf.mjs`, `docs/open-vocab-3dgs-imagine-pipeline-paper.pdf`	KaTeX/Marked/Chromium renderer and generated PDF

Appendix B. Operator one-liner

```
python3 -m http.server 8090 --bind 127.0.0.1 &
./semantic_sidecar/launch.sh &
./gaussian_splatting_webgpu/launch-webgpu-chrome.sh
```